

FACULTAD DE INGENIERÍA
Universidad de Buenos Aires

75.29 / 95.06

Teoría de Algoritmos 1 - 1c 2025

Trabajo Práctico N° 2

ACORN

Fecha de Entrega:

7 de julio de 2025

Integrantes:

Alumno	Padrón
Rogger Aldair Paredes Tavera	97976
Santiago Nahuel Ruiz Sugliani	106768
Maria Mercedes Slepowron Majowiecki	109454
Jose Ignacio Adelardi	111701

Parte 1: Personal para Proyectos.....	3
1. Expresar y explicar el problema como un problema de programación lineal.....	3
2. Explicar cómo resolver el problema utilizando Simplex. Brindar pseudocódigo y explicación de su propuesta. Expresar como se representa la solución.....	3
Pseudocódigo.....	4
3. Analizar la complejidad temporal y espacial de cada uno de los pasos de su solución.....	4
Complejidad Temporal.....	4
Complejidad Espacial.....	5
4. Presente paso a paso la solución del problema. Presente esquemáticamente cada iteración del algoritmo. Indique qué decisiones se toma en cada uno de estos y los cálculos realizados.....	5
5. Analizar la solución obtenida. ¿Corresponde a la solución óptima del problema? Según la naturaleza del problema, ¿lo puede encontrar en tiempo polinomial? Justificar.....	7
6. Investigar la aplicación de Branch and Bound dentro de programación lineal entera. Explique cómo funciona paso a paso. Brinde pseudocódigo. ¿Lo puede utilizar en el problema? En caso afirmativo desarrolle como hacerlo paso a paso.....	7
7. Programe su propuesta. Puede utilizar librerías para Simplex.....	10
Parte 2: Buscando una solución aceptable.....	11
Para el desarrollo de un proyecto estamos buscando inversores. Contamos con una preselección de “n” de estos. cada uno de ellos nos prometió aportar cierta cantidad de dinero (este valor no necesariamente es igual para cada inversor). Nos gustaría aceptar a todos. Sin embargo, algunos inversores se niegan a participar si otros están. Nos gustaría determinar a cuáles aceptar para maximizar el dinero total para la inversión.....	11
1. Analizar el problema y determinar si es posible resolverlo de forma eficiente.....	11
2. Proponer 2 algoritmos heurísticos/randomizados o de aproximación para resolverlo. Explicar cómo funcionan.....	11
3. Brindar pseudocódigo y análisis completo del mismo (análisis de complejidad temporal, espacial, optimalidad, grado de aproximación, según corresponda al tipo de solución).....	12
4. Realizar su programa y brindar varios ejemplos para su ejecución.....	17
5. Realizar un análisis empírico de sus soluciones. ¿Puede determinar en qué casos es mejor uno que el otro?.....	20
Evaluación comparativa de Simulated Annealing y Greedy.....	20
Cambio de parámetros en Simulated Annealing.....	20
Resultados obtenidos.....	21
Conclusión final.....	21
Parte 3: Autómatas Finitos.....	22
1. Construir un autómata finito determinista (AFD) de forma formal para cada uno de los lenguajes simples.....	22
2. Basándose en los autómatas construidos generar de forma formal los dos autómatas finitos determinísticos de los lenguajes más complejos.....	25
3. Representar cada uno de los AFD mediante su representación gráfica.....	27
4. Para cada AFD Brindar un ejemplo de una cadena que acepta y otra que rechaza. Explicar cómo determinar su autómata si es aceptado o rechazado paso a paso.....	27
Bibliografía.....	30

Parte 1: Personal para Proyectos

1. Expresar y explicar el problema como un problema de programación lineal.

El objetivo del problema es: Determinar cuántos proyectos y de qué tipo (A, B o C) debe aceptar la consultora para maximizar las ganancias totales, respetando la disponibilidad limitada de ingenieros y diseñadores.

Variables de decisión

x_A : cantidad de proyectos tipo A a realizar

x_B : cantidad de proyectos tipo B a realizar

x_C : cantidad de proyectos tipo C a realizar

Función objetivo

Debemos Maximizar las ganancias en la elección de proyectos:

$$\text{Maximizar } Z = 200x_A + 100x_B + 300x_C$$

Restricciones

Cada tipo de proyecto usa: • A: 1 ingeniero • B: 1 ingeniero • C: 3 ingenieros Entonces: $x_A + x_B + 3x_C \leq 17$	Cada tipo de proyecto usa: • A: 3 diseñadores • B: 2 diseñadores • C: 2 diseñadores Entonces: 3 $3x_A + 2x_B + 2x_C \leq 11$	Restricciones de no negatividad: $0 \leq x_A$ $0 \leq x_B$ $0 \leq x_C$
--	---	---

2. Explicar cómo resolver el problema utilizando Simplex. Brindar pseudocódigo y explicación de su propuesta. Expresar como se representa la solución.

Para transformar a SIMPLEX tenemos que pasar a la forma de holgura

$$\text{Maximizar } Z = 200x_A + 100x_B + 300x_C$$

$$x_D = 17 - x_A - x_B - 3x_C$$

$$x_E = 11 - 3x_A - 2x_B - 2x_C$$

$$0 \leq x_A, 0 \leq x_B, 0 \leq x_C, 0 \leq x_D, 0 \leq x_E$$

- Tenemos $N=2$ variables de decisión y $M=3$ restricciones
- Para resolver simplex tenemos 2 variables básicas: x_A, x_B, x_C variables libres: x_D, x_E
- Expresamos una solución como $(x_A, x_B, x_C, x_D, x_E)$

Pseudocódigo

función SIMPLEX(función_objetivo, restricciones, variables_decisión, variables_holgura):

Mientras exista un coeficiente positivo en la función objetivo:

 Seleccionar la variable entrante ve con el mayor coeficiente positivo

 Para cada restricción:

 Si el coeficiente de $ve > 0$:

 Calcular el ratio: término independiente / coeficiente de ve

 Si no hay ratios válidos:

 Retornar “problema ilimitado”

 Seleccionar la variable saliente vs con el menor ratio

 Realizar pivot:

 Despejar ve en la ecuación de vs

 Reemplazar ve en el resto de las restricciones y en la función objetivo

 Actualizar las variables básicas y no básicas

Retornar los valores óptimos de las variables de decisión,
el valor de la función objetivo y el sistema final de restricciones

función MAIN():

Transformar el problema a forma de holgura agregando variables de holgura

Inicializar la solución básica:

 variables_decisión en 0

 variables_holgura según los términos independientes

$Z = 0$

Llamar a **SIMPLEX** con la función objetivo y restricciones transformadas

Retornar la solución óptima

3. Analizar la complejidad temporal y espacial de cada uno de los pasos de su solución.

Complejidad Temporal

- Transformar a forma de holgura:

- Se agregan m variables (una por cada restricción) y se reescriben las ecuaciones.
- Tiempo: $O(m * n)$
- Selección de variable a aumentar:
 - Se busca el coeficiente positivo más grande en la función objetivo.
 - Tiempo: $O(n)$
- Determinación de variable que ajusta:
 - Se evalúan las m restricciones para ver cuál impide seguir aumentando.
 - Tiempo: $O(m)$
- Pivot (actualización del sistema):
 - Se despeja la variable que entra y se reemplaza en todas las restricciones y en la función objetivo.
 - Tiempo por iteración: $O(m * n)$
- Número de iteraciones:
 - En el peor caso, se puede recorrer cada punto extremo del politopo.
 - Cantidad de vértices posibles: $O\left(\binom{n+m}{n}\right)$ Esto corresponde a un proceso no polinomial Aunque en la práctica la mayoría de los problemas se suelen resolver de forma rápida. La complejidad temporal final en el peor de los casos será $O(2^n * m)$

Complejidad Espacial

- Se almacenan n variables de decisión y m de holgura.
- $O(m)$ por variables de holgura
- $O(n + m)$ para registrar las variables básicas/libres
- $O(n * m)$ para la matriz de coeficientes
- Espacio total requerido: $O(n * m)$

4. Presente paso a paso la solución del problema. Presente esquemáticamente cada iteración del algoritmo. Indique qué decisiones se toma en cada uno de estos y los cálculos realizados.

Para transformar a SIMPLEX tenemos que pasar a la forma de holgura

$$\text{Maximizar } Z = 200x_A + 100x_B + 300x_C$$

$$x_D = 17 - x_A - x_B - 3x_C$$

$$x_E = 11 - 3x_A - 2x_B - 2x_C$$

$$0 \leq x_A, 0 \leq x_B, 0 \leq x_C, 0 \leq x_D, 0 \leq x_E$$

- Tenemos $N=2$ variables de decisión y $M=3$ restricciones
- Para resolver simplex tenemos 2 variables básicas: x_A, x_B, x_C variables libres: x_D, x_E

- Expresamos una solución como $(x_A, x_B, x_C, x_D, x_E)$

Solución básica: $x_A = 0, x_B = 0, x_C = 0$

$$Z = 200x_A + 100x_B + 300x_C = 0$$

$$x_D = 17 - x_A - x_B - 3x_C = 17$$

$$x_E = 11 - 3x_A - 2x_B - 2x_C = 11$$

$(0, 0, 0, 17, 11)$

Determinación de variable entrante y saliente, elegimos x_C que multiplicada por 300 es la que aumenta mas el valor objetivo

$$x_D: 17 - 0 - 0 - 3x_C = 0 \rightarrow x_C = 5.7$$

$x_E: 11 - 0 - 0 - 2x_C = 0 \rightarrow x_C = 5.5 \leftarrow$ queda ajustada, con lo que la variable libre que queda ajustada es $x_E = 0$, la cual será la variable entrante en el proceso de pivot y x_C será la variable saliente con valor 5.5 en el mismo, con lo cual la nueva solución será $(0, 0, 5.5, 0.5, 0) \rightarrow$ lo que corresponde a $Z = 200x_A + 100x_B + 300x_C = 1650$ para $(0, 0, 5.5)$

Proceso Pivot despejamos x_C de la ecuación de restricción en la que x_E quedó ajustada.

$$x_E = 11 - 3x_A - 2x_B - 2x_C \rightarrow x_C = 5.5 - 1.5x_A - x_B - 0.5x_E$$

Reemplazamos x_C en el resto de las restricciones y función objetivo

$$\text{Maximizar } Z = 1650 - 150x_A - 250x_B - 200x_E$$

Sujeto a:

$$x_C = 5.5 - 1.5x_A - x_B - 0.5x_E$$

$$x_D = 0.5 + 3.5x_A + 2x_B - 1.5x_E$$

Corresponde a un problema equivalente al anterior Con x_A, x_B, x_E como variables básicas y x_C, x_D variables libres.

No quedan coeficientes positivos en la función objetivo con lo que no se puede mejorar aumentando variables su valor.

Finalizamos el algoritmo con la solución óptima $(x_A, x_B, x_C) = (0, 0, 5.5)$. Y valor objetivo 1650

5. Analizar la solución obtenida. ¿Corresponde a la solución óptima del problema? Según la naturaleza del problema, ¿lo puede encontrar en tiempo polinomial? Justificar.

De acuerdo con el método Simplex, el algoritmo termina cuando no existen más variables en la función objetivo con coeficientes positivos, lo cual indica que ya no se puede mejorar el valor de Z. Esto significa que:

La solución encontrada corresponde a un punto extremo del politopo de soluciones factibles y es óptima.

Esto se justifica teóricamente por el hecho de que en problemas de programación lineal, la solución óptima (si existe) siempre se alcanza en uno de los vértices del conjunto factible.

En el peor de los casos, el algoritmo Simplex no garantiza tiempo polinomial, ya que puede recorrer un número exponencial de vértices (hasta $\binom{n+m}{n}$).

Sin embargo, en la práctica:

El algoritmo Simplex suele encontrar la solución óptima en pocas iteraciones y se comporta de forma eficiente.

En este caso particular, la solución se alcanzó en una sola iteración, por lo que el tiempo de ejecución fue muy bajo (prácticamente lineal).

6. Investigar la aplicación de Branch and Bound dentro de programación lineal entera. Explique cómo funciona paso a paso. Brinde pseudocódigo. ¿Lo puede utilizar en el problema? En caso afirmativo desarrolle como hacerlo paso a paso.

Existe una gran cantidad de problemas como el que se nos presenta en este apartado donde se requieren tomar decisiones que involucren variables enteras. En nuestro caso, tenemos una consultora que ofrece personal para diferentes tipos de proyectos, y no sería coherente aceptar, por ejemplo, 5.5 proyectos del tipo C. Este tipo de problemas no pueden resolverse adecuadamente utilizando solo programación lineal continua. Para ello, se empleó la programación lineal entera, donde uno de sus métodos más utilizados para resolver problemas es Branch & Bound.

Siguiendo la propuesta de Taylor (2007, p.9), el método de Branch and Bound consiste en:

1. resolver el problema relajado (sin restricciones de enteros)
2. en el nodo 1, tomar la solución relajada como cota superior y la solución entera (redondeada hacia abajo) como cota inferior.
3. si no es entera, se elige la variable con mayor parte fraccionaria para ramificar
4. se generan dos subproblemas: uno con la variable menor o igual al piso del valor, y otro con la variable mayor o igual al techo

5. Se resuelve el modelo de programación lineal relajado en cada uno de estos nuevos nodos con la restricción añadida.
6. se realiza una poda: Se descartan subproblemas en caso de que sean inviables, su valor objetivo relajado no supere a la mejor solución entera actual, o la solución sea entera pero no mejor a la máxima encontrada
7. Se elige el siguiente nodo a explorar, siguiendo estrategias como BFS o DFS. Si el proceso produce una solución entera factible con el mayor valor de cota superior entre todos los nodos finales, entonces se ha alcanzado la solución entera óptima. Si no aparece, una solución entera factible, se ramifica desde el nodo con la mayor cota superior.
8. Se repite este proceso desde el paso 3 hasta que no haya más nodos por explorar.

Para un modelo de minimización, las soluciones relajadas se redondean hacia arriba, y las cotas superior e inferior se invierten.

Pseudocódigo para Branch & Bound aplicado en Programación Lineal Entera

```
BranchAndBound(P):  
  cola ← {P relajado}  
  mejor_Z ←  $-\infty$   
  mejor_sol ←  $\emptyset$   
  
  mientras cola no vacía:  
    P_act ← desencolar(cola)  
    sol ← resolver_relajado(P_act)  
  
    si sol no factible: continuar  
  
    si sol entera y  $Z(sol) > mejor\_Z$ :  
      mejor_Z ←  $Z(sol)$   
      mejor_sol ← sol  
      continuar  
  
    si  $Z(sol) \leq mejor\_Z$ : continuar # podemos  
  
    v_frac ← variable con mayor parte fraccionaria  
    cola.insertar(P_act + (v_frac ≤ piso(v_frac)))  
    cola.insertar(P_act + (v_frac ≥ techo(v_frac)))  
  
  retornar mejor_sol
```

Aplicación en nuestro problema y paso a paso

En nuestro problema de selección de proyectos, podemos aplicar Branch & Bound si exigimos que las soluciones (los proyectos seleccionados) sean enteros. Es decir, que no se puedan realizar fracciones de proyectos.

Buscamos maximizar $Z = 200x_A + 100x_B + 300x_C$ con las siguientes restricciones para los ingenieros, los diseñadores y la no negatividad:

$$x_D = 17 - x_A - x_B - 3x_C$$

$$x_E = 11 - 3x_A - 2x_B - 2x_C$$

$$0 \leq x_A, 0 \leq x_B, 0 \leq x_C, 0 \leq x_D, 0 \leq x_E$$

1. Resolvemos el problema relajado

Utilizamos Simplex y vamos a obtener $x_A = 0, x_B = 0, x_C = 5.5$

$$Z = 200 * 0 + 100 * 0 + 300 * 5.5 = 1650$$

Como la solución no es entera, aplicamos Branch & Bound.

2. Ramificamos sobre $x_C = 5.5$

Vamos a crear dos subproblemas, un nodo izquierdo con $N1 < 5$ y un nodo derecho $N2 \geq 6$.

Subproblema $N1 < 5$. Vamos a contar con las siguientes restricciones:

$$x_D = 17 - x_A - x_B - 3x_C$$

$$x_E = 11 - 3x_A - 2x_B - 2x_C$$

$$x_C \leq 5$$

$$0 \leq x_A, 0 \leq x_B, 0 \leq x_C, 0 \leq x_D, 0 \leq x_E$$

Resolvemos el problema relajado y obtenemos: $x_A = 0, x_B = 0, x_C = 5$, una solución entera factible con $Z = 200 * 0 + 100 * 0 + 300 * 5 = 1500$.

Subproblema $N1 \geq 6$. Vamos a contar con las siguientes restricciones:

$$x_D = 17 - x_A - x_B - 3x_C$$

$$x_E = 11 - 3x_A - 2x_B - 2x_C$$

$$x_C \geq 6$$

$$0 \leq x_A, 0 \leq x_B, 0 \leq x_C, 0 \leq x_D, 0 \leq x_E$$

Pero cuando intentamos resolver el problema para este valor de C, resulta que violamos la restricción de x_D y la de x_E , pues con $x_C = 6, 3 * 6 = 18$ y vamos a tener

$$x_D = 17 - x_A - x_B - 18 \text{ con } x_D \leq 0.$$

$$\text{Para } x_E \text{ tenemos } x_E = 11 - 3x_A - 2x_B - 2x_C = 11 - 3x_A - 2x_B - 12 \leq 0.$$

Dado que este es un nodo que viola las restricciones del sistema, lo podamos.

El problema original permitía soluciones fraccionarias, obteniendo con el método Simplex una ganancia máxima de 1650 millones de pesos al aceptar 5.5 proyectos tipo C. Sin embargo, en un escenario realista, los proyectos deben aceptarse de forma entera. Por lo que fue necesario aplicar una técnica que garantice variables enteras.

El algoritmo Branch and Bound permitió realizar la exploración de dicho espacio de soluciones enteras. Mediante la ramificación sobre la variable fraccionaria ($C = 5.5$) y la aplicación de poda en las ramas que violaran las restricciones, alcanzamos la solución óptima entera con:

- Proyectos A: 0
- Proyectos B: 0
- Proyectos C: 5

Máxima ganancia alcanzable: $Z = 5 \times 300 = 1500$ millones de pesos

7. Programe su propuesta. Puede utilizar librerías para Simplex.

Para resolver el problema utilizamos la librería PuLP. El programa **simplex.py** resuelve el problema original de maximización y al finalizar retorna el valor objetivo óptimo y los valores óptimos de cada variable de decisión. El output se ve así:

Estado: 1, Optimal

$$x_A = 0.0$$

$$x_B = 0.0$$

$$x_C = 5.5$$

Ganancia total = 1650.0

Para ejecutar el programa **simplex.py** asegurarse de tener instalada la librería PuLP con `pip install pulp`. Luego, correr el programa con `python simplex.py`

Parte 2: Buscando una solución aceptable

Para el desarrollo de un proyecto estamos buscando inversores. Contamos con una preselección de “n” de estos. cada uno de ellos nos prometió aportar cierta cantidad de dinero (este valor no necesariamente es igual para cada inversor). Nos gustaría aceptar a todos. Sin embargo, algunos inversores se niegan a participar si otros están. Nos gustaría determinar a cuáles aceptar para maximizar el dinero total para la inversión.

1. Analizar el problema y determinar si es posible resolverlo de forma eficiente.

Podríamos modelar este problema mediante un grafo $G=(V, E)$ no dirigido y sin pesos donde V está compuesto por los inversores (es decir que cada inversor es un vértice en V) y E está compuesto por las enemistades (es decir que si el inversor A se niega a participar si el inversor B participa entonces existe la arista (A, B) en E). Al buscar la máxima inversión de modo que no exista un par de inversores “enemistados” seleccionados en simultáneo para participar, lo que queremos encontrar es el conjunto independiente cuyos vértices otorguen la mayor ganancia posible. Sabemos que el problema del conjunto independiente es NP-Completo y el modelado que dimos sobre este problema no es más que una variante del mismo donde no buscamos que el conjunto tenga la máxima cantidad de vértices posibles, sino que los vértices seleccionados otorguen la mayor ganancia posible. Por lo tanto podemos asumir que también es NP-Completo y dicho esto no podemos decir que es posible resolverlo de manera eficiente ya que eso significa que podemos resolver eficientemente el problema del conjunto independiente y por lo tanto que $P = NP$.

2. Proponer 2 algoritmos heurísticos/randomizados o de aproximación para resolverlo. Explicar cómo funcionan

1. Algoritmo heurístico/randomizado basado en “Simulated Annealing”

La idea es replicar el enfoque mencionado a partir de un conjunto solución compuesto por un único inversor el cual compone una solución factible. En cada iteración evaluaremos un estado vecino S_v que dado S el estado actual, consista en agregar un inversor a S o quitar uno (este tipo de operación será elegida de forma aleatoria), esto lo haremos con cuidado dado que se debe preservar que S_v sea una solución factible y por ende al quitar un inversor no debemos realizar ninguna comprobación, pero al agregar uno debemos corroborar que el mismo no se encuentre “enemistado” con alguno de los inversores de S . Mantendremos actualizado a lo largo de las iteraciones un conjunto S_c el cual contendrá los vértices que no se encuentran en S . Por lo tanto al eliminar un inversor aleatorio de S lo sumaremos a S_c , al agregar un inversor aleatorio el mismo será seleccionado de S_c y se realizará la comprobación mencionada.

Como podemos notar, en el modelado de este problema haciendo uso de “Simulated Annealing” simplemente se observa en la elección de un nuevo estado vecino, y en consecuencia, el cálculo de la función costo (denotada como FC en el pseudocódigo) se basa en las ganancias que aportan los inversores de un estado (en este caso un conjunto de inversores que componen una solución factible al problema)

2. Algoritmo heurístico basado en densidad conflicto-aporte

Este algoritmo se basa en una estrategia greedy que toma decisiones locales en función de una medida de “densidad” que busca equilibrar el beneficio que aporta un inversor con la cantidad de conflictos que genera. El problema se modela como un grafo no dirigido, donde cada vértice representa un inversor y una arista entre dos vértices indica que esos dos inversores no pueden formar parte de la solución al mismo tiempo. El objetivo es encontrar un conjunto independiente de vértices (es decir, un subconjunto tal que ningún par de sus miembros esté conectado por una arista) que maximice la suma de los aportes económicos de los inversores incluidos.

Para guiar la selección, a cada inversor se le asigna un valor de “densidad”, definido como el cociente entre el dinero que aporta y su grado en el grafo. Así, la fórmula utilizada es: $\text{densidad}(v) = \text{aporte}(v) / (\text{grado}(v) + 1)$. Cuanto mayor sea esta densidad, mayor es el beneficio relativo del inversor respecto a sus conflictos.

Una vez calculadas estas densidades, los inversores se ordenan de forma decreciente según ese valor. Luego, se recorren en ese orden, y cada inversor es agregado al conjunto solución solo si no entra en conflicto con ninguno de los inversores ya seleccionados. Este procedimiento se repite hasta haber evaluado a todos los inversores. Al finalizar, se devuelve el conjunto construido y la suma total de los aportes de sus miembros.

3. Brindar pseudocódigo y análisis completo del mismo (análisis de complejidad temporal, espacial, optimalidad, grado de aproximación, según corresponda al tipo de solución).

1) Pseudocódigo del algoritmo heurístico/randomizado basado en “Simulated Annealing”:

```
Sea inversores un arreglo de largo “n” de la forma (inversor, inversion, enemigos)
Sea To la temperatura inicial
Sea M la cantidad máxima de iteraciones
Sea C la constante de enfriamiento

OBTENER_MEJORES_INVERSORES(inversores)
  G, ganancia = CONSTRUIR_GRAFO(inversores)
  S_Max = SIMULATED_ANNEALING(inversores, G, ganancia)
  Retornar S_Max, FC(S_Max, ganancia)

CONSTRUIR_GRAFO(inversores)
```

Creamos el grafo $G=(V, E)$ no dirigido y sin pesos.

Creamos el diccionario ganancias.

Por cada inv en inversores

Sea inversor = inv[0]

Sea ganancia = inv[1]

Agregar nodo inversor a G

Agregar la entrada ganancias[inversor] = ganancia

Por cada inv en inversores

Sea inversor = inv[0]

Sea enemigos = inv[2] el arreglo de inversores enemigos

Por cada e en enemigos

Agrego la arista (inversor, e) a G

Retorno G, ganancias

SIMULATED_ANNEALING(inversores, G, ganancias)

Sea Vertices el arreglo de vértices de G

Sea V un vértice aleatorio de Vértices

Sea S_Max el conjunto de inversores sin enemistades con mayor ganancia encontrado al momento (inicialmente vacío)

Sea S el conjunto inicial de inversores compuesto únicamente por V (estado inicial)

Sea Sc el conjunto de inversores que no contiene a V

Sea it = 0 la cantidad de iteraciones realizadas

Sea T = To

Establecer S_Max como el estado inicial aleatorio S

Mientras it < M

Sv = ELEGIR_ESTADO_VECINO(S, Sc, G)

S = ELEGIR_NUEVO_ESTADO_ACTUAL(T, S, Sv, ganancia)

Establecer S_Max como el estado de costo máximo entre FC(S_Max, ganancias) y FC(S, ganancias)

Realizar la disminución de la temperatura $T = T * C$

Incrementar it en 1

Retornar S_Max

FC(Estado, ganancias)

Retornar Sumatoria(ganancias[Wi] para i en Estado)

ELEGIR_ESTADO_VECINO(EstadoActual, Sc, G)

Sv = copy(EstadoActual)

Op = Random(Quitar, Agregar)

Si Op == Quitar y hay vértices por quitar de Sv

Quitar de Sv un vértice aleatorio

Agregar a Sc el vértice que quitamos de Sv

```
Si Op == Agregar y hay vértices por agregar
    Agregar a Sv un vértice aleatorio de G preservando factibilidad
    Quitar de Sc el vértice que agregamos a Sv
Retornar Sv, Sc
```

```
ELEGIR_NUEVO_ESTADO_ACTUAL(T, Sv, S)
```

```
    Calcular C el valor de FC(S, ganancia)
    Calcular Cv el valor de FC(Sv, ganancia)
```

```
    Si Cv > C
        Retornar Sv
```

```
    Calcular R aleatorio entre 0 y 1
    Calcular exp = (Cv - C) / T
```

```
    Si R < eexp
        Retornar Sv
    Retornar S
```

Análisis del algoritmo

Complejidad temporal

- El algoritmo consta de tres partes y ellas son la construcción del grafo, la selección de los inversores y las R ejecuciones de Simulated Annealing en búsqueda de la mejor selección de inversores, procederemos a detallar cada una de ellas.
- Para la construcción del grafo contamos con una lista de N inversores de la forma (inversor, inversión, enemigos), en primer lugar vamos a recorrer los inversores y por cada uno de ellos agregar un vértice al grafo además de guardar en un diccionario la ganancia que significa seleccionar a este inversor, esto representa una complejidad $O(N)$, en segundo lugar volvemos a recorrer a los N inversores tomando la lista de enemigos en cada iteración para el inversor actual para entonces crear las aristas que unen a un inversor con cada enemigo del mismo. Finalmente la complejidad temporal es $O(N + N^2)$ dado que un inversor puede estar enemistado con los N-1 demás inversores, y por lo tanto la complejidad temporal de la función CONSTRUIR_GRAFO es $O(N + N^2)$.
- Respecto a la selección de inversores, comenzamos instanciando las variables a utilizar a lo largo del algoritmo. Posteriormente seleccionaremos un inversor aleatorio en $O(1)$ el cual será nuestra solución inicial. Luego comienzan las iteraciones del algoritmo donde debemos:

1) Elegir un estado vecino a partir del actual.

Se realizan dos elecciones aleatorias, la primera sobre el tipo de operación por llevar a cabo (que se realiza en $O(1)$) al igual que la segunda la cual se basa en elegir aleatoriamente el vértice V a tratar (tanto al agregar como al quitar). La eliminación de tal vértice V del conjunto Sv se realiza en tiempo $O(1)$ mientras que el agregado debe contemplar la factibilidad de realizar tal acción, con lo cual ninguno de los adyacentes de V debe ser un inversor en Sv y representando entonces una complejidad de $O(N)$.

2) Seleccionar entre el estado vecino y el actual con cuál vamos a quedarnos.

Esta selección se reduce al cálculo de la función costo para S y S_v sumado a operaciones matemáticas de tiempo $O(1)$, por lo cual la complejidad de FC será determinante aquí. Esta función se calcula recorriendo todos los inversores para el estado en cuestión y consultando en el diccionario de ganancias previamente generado, con lo cual la complejidad es $O(N)$ dado que en el mejor caso el grafo no tiene aristas y puedo agregar todos los inversores.

3) Actualización de las variables utilizadas

Se decrementa la temperatura T para la siguiente iteración al multiplicarla por el factor de enfriamiento C , además se incrementa el contador de iteraciones, con lo cual tenemos operaciones de tiempo constante.

- Finalmente, en conjunto realizamos la construcción del grafo más la selección aleatoria un total de M veces, por lo tanto y considerando la complejidad indicada para esta selección, la complejidad es $O(N + N^2) + O(M * N)$.
- Finalmente la complejidad total del algoritmo es $O(N^2 + M * N)$.

Complejidad espacial

- Para determinar la complejidad espacial debemos tener en cuenta al grafo que debemos construir más las estructuras que mantenemos a lo largo del algoritmo. Para el caso del grafo tenemos una complejidad $O(N^2)$ siendo N la cantidad total de inversores, a lo sumo cada inversor se encuentra enemistado con todos los demás. Respecto a las estructuras auxiliares utilizadas a lo largo del algoritmo contamos con una complejidad $O(N)$ dada por S, S_c, S_v que se tratan de conjuntos que contienen elementos que representan los vértices de G (las demás variables ocupan un espacio $O(1)$ ya que son valores numéricos o booleanos). Finalmente la complejidad espacial es $O(N^2 + N) = O(N^2)$.

Optimalidad

- No podemos afirmar que sea óptimo ya que Simulated Annealing es un algoritmo heurístico que busca encontrar soluciones cercanas a la óptima con una complejidad polinomial. La exploración del espacio de estados posibles depende de la elección sobre la temperatura inicial T_0 , el factor de enfriamiento C y el número de iteraciones M como también de la solución inicial la cual en este caso consta de un único vértice (siendo que también puede partir de un conjunto de vértices no adyacentes) y la manera de generar un estado vecino a partir del actual. Mediante una configuración adecuada sobre T_0, C y M a partir de una serie de pruebas es que podemos encontrar una solución cercana al óptimo para cualquier caso.

2) Pseudocódigo del Algoritmo heurístico basado en densidad conflicto-aporte

CONSTRUIR_GRAFO(inversores) se mantiene igual al detallado en el algoritmo anterior

GREEDY_DENSIDAD(G , ganancias)

Paso 1: calcular densidades

Para cada v en $G.V$

densidad[v] = ganancias[v] / (grado(v) + 1)

```
# Paso 2: ordenar vértices por densidad descendente
L = lista con los vértices de G ordenados por densidad no creciente

# Paso 3: recorrido greedy
seleccionados = conjunto vacío
beneficio_total = 0

Para cada v en L
    es_compatible = verdadero
    Para cada w en G.adyacentes(v)
        Si w ∈ seleccionados
            es_compatible = falso
            salir del for interno

    Si es_compatible
        agregar v a seleccionados
        beneficio_total = beneficio_total + ganancias[v]

devolver beneficio_total , seleccionados

OBTENER_SOLUCION_GREEDY(inversores)
    G , ganancias = CONSTRUIR_GRAFO(inversores)
    beneficio , seleccionados = GREEDY_DENSIDAD(G , ganancias)
    devolver beneficio , seleccionados
```

Análisis del algoritmo

- Complejidad temporal

La complejidad temporal del algoritmo es $O(n \log n + m)$, donde n es la cantidad de inversores y m la cantidad de conflictos (aristas en el grafo). Primero, calcular las densidades de los vértices implica recorrer todos los nodos y obtener su grado, lo que puede hacerse eficientemente si el grafo está representado mediante listas de adyacencia, en tiempo $O(n + m)$. Luego, ordenar los vértices por densidad descendente toma $O(n \log n)$. Finalmente, el recorrido greedy revisa cada vértice en orden y compara con sus vecinos para verificar compatibilidad, lo que en total también se hace en $O(n + m)$. Así, la suma total de pasos es dominada por la ordenación, quedando la complejidad final en $O(n \log n + m)$, que es eficiente y aceptable para instancias grandes.

- Complejidad espacial

La complejidad espacial del algoritmo es $O(n + m)$. Esto se debe a que se almacena el grafo como una estructura de listas de adyacencia, lo cual requiere espacio proporcional al número de vértices y aristas. Además, se utilizan estructuras auxiliares como el diccionario de ganancias, el arreglo de densidades, la lista de ordenamiento y el conjunto de inversores seleccionados, todas de tamaño proporcional a n . En consecuencia, el algoritmo no requiere espacio adicional significativo más allá del necesario para representar la instancia, y su uso de memoria es eficiente.

- Optimalidad

El algoritmo no garantiza una solución óptima. Se trata de una heurística voraz (greedy), que toma decisiones locales basadas en un criterio de densidad beneficio/conflicto, sin considerar el impacto global de esas decisiones. En algunos casos, esto puede llevar a excluir combinaciones de inversores que, si se hubieran evaluado en conjunto, habrían dado un resultado con mayor ganancia total. Por lo tanto, la solución que devuelve el algoritmo puede estar lejos de la óptima, especialmente en instancias con muchas interacciones entre los nodos. Sin embargo, este comportamiento es típico de las heurísticas, y no invalida su uso cuando se busca eficiencia computacional en lugar de exactitud.

- Grado de aproximación

El grado de aproximación del algoritmo no está garantizado de forma formal como una cota fija general, pero se puede decir que en el peor caso logra un beneficio que representa al menos una fracción $1 / (\Delta + 1)$ del óptimo, donde Δ es el grado máximo del grafo. Esta cota proviene de análisis clásicos de heurísticas similares en problemas de conjuntos independientes ponderados. Si bien esto no asegura una buena aproximación en todos los casos (por ejemplo, en grafos densos), en grafos donde Δ es pequeño el algoritmo puede acercarse razonablemente al valor óptimo. En la práctica, el desempeño suele ser mucho mejor que esta cota teórica, especialmente si el conflicto entre inversores no es demasiado alto.

4. Realizar su programa y brindar varios ejemplos para su ejecución

Para ejecutar cada uno de los algoritmos propuestos se deberá especificar cuál se quiere utilizar además de la instancia del problema sobre la cual se ejecutará el mismo. Esta instancia se encontrará definida en un archivo de texto donde cada línea será de la forma:

INVERSOR, GANANCIA, E1, E2, E3, ..., EN

- **INVERSOR** será el identificador de un inversor.
- **GANANCIA** será el monto que aporte ese inversor al ser seleccionado.
- **E1, E2, E3, ..., EN** serán los inversores con los cuales **INVERSOR** se encuentra enemistado (puede no estar enemistado con absolutamente ningún inversor).

La manera de ejecutar estos algoritmos propuestos con una instancia del problema es la siguiente:

\$ python3 <nombre_archivo_algoritmo> <nombre_archivo_instancia_problema>

Por ejemplo para hacer uso del algoritmo que implementa Simulated Annealing para los archivos de los siguientes ejemplos, sus respectivas invocaciones por consola son las indicadas en cada caso.

Ejemplo 1

Veamos un el contenido de un archivo de ejemplo con tres inversores donde el primero se encuentra enemistado con los otros dos y estos últimos dos no se encuentran enemistados entre ellos:

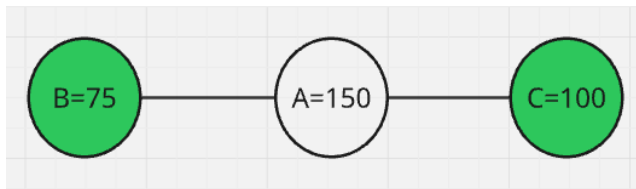
A,150,B,C

B,75

C,100

\$ python3 propuesta_sa.py ejemplo_1_informe.txt

El grafo resultante del procesamiento de este archivo será:



Por lo tanto la solución óptima al mismo es la elección de los inversores B y C, obteniendo una ganancia total de 175 la cual supera a solo elegir A que representaría una ganancia de 150 (y como se puede observar en este caso corresponde al conjunto independiente más grande además del de mayor ganancia).

Ejemplo 2

A,350,B,C,D,E

B,175,A,E

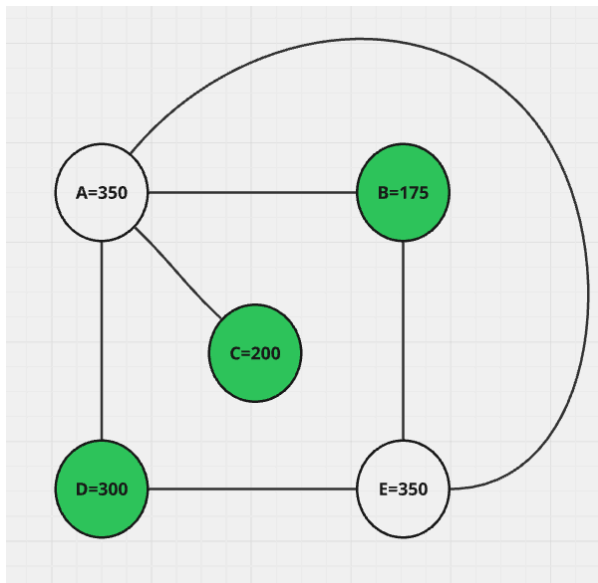
C,200,A

D,300,A,E

E,350,A,B,D

\$ python3 propuesta_sa.py ejemplo_2_informe.txt

Para este ejemplo podemos observar que la mayor ganancia la otorga la elección de los inversores B, C y D (al igual que en el ejemplo anterior, los mismos conforman el mayor conjunto independiente en tal grafo).



Ejemplo 3

A,1000,B,C,D,E

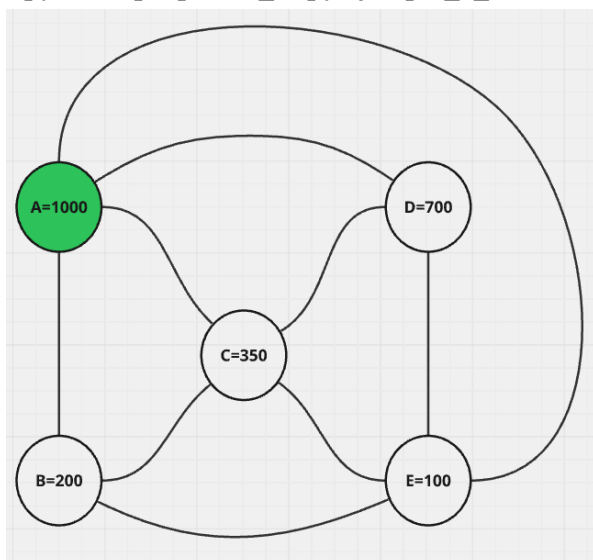
B,175,A,C,E

C,200,A,B,D,E

D,300,A,C,E

E,350,A,B,C,D

\$ python3 propuesta_sa.py ejemplo_3_informe.txt



Para este caso podemos observar que el óptimo es elegir solo al inversor A, teniendo una ganancia total de 1000. Además, este caso es distinto a los anteriores en función de la cantidad de inversores en el conjunto independiente dentro de la solución ya que {B, D} conforma un conjunto independiente de mayor cardinalidad pero de una ganancia total de 900.

5. Realizar un análisis empírico de sus soluciones. ¿Puede determinar en qué casos es mejor uno que el otro?

Para el análisis empírico se ejecutó: `python ejecutar_todas_las_instancias.py`

Con que obtuvimos los siguientes resultados.

Archivo	Ganancia_greedy	Ganancia_sa	Mejor	Empate	Tiempo_greedy_ms	Tiempo_sa_ms
instancias\instancia_n100_p10_r0.txt	1912	1384	Greedy	FALSE	0.83	24.76
instancias\instancia_n100_p20_r0.txt	1093	1216	SA	FALSE	1.46	22.09
instancias\instancia_n100_p30_r0.txt	845	613	Greedy	FALSE	1.89	17.99
instancias\instancia_n10p010r0.txt	370	370	Greedy	TRUE	0.05	11.39
instancias\instancia_n10p030r0.txt	326	309	Greedy	FALSE	0.05	9.18
instancias\instancia_n20p005r0.txt	955	977	SA	FALSE	0.05	17.42
instancias\instancia_n20p020r0.txt	568	419	Greedy	FALSE	0.11	11.01
instancias\instancia_n30p010r0.txt	1034	859	Greedy	FALSE	0.09	17.42
instancias\instancia_n30p025r0.txt	542	449	Greedy	FALSE	0.21	11.36
instancias\instancia_n50_p10_r0.txt	1134	857	Greedy	FALSE	0.23	17.47
instancias\instancia_n50_p20_r0.txt	835	951	SA	FALSE	0.38	16.28
instancias\instancia_n50_p30_r0.txt	679	532	Greedy	FALSE	0.97	13.63
instancias\instancia_n60_p10_r0.txt	1333	1130	Greedy	FALSE	0.31	21.04
instancias\instancia_n60_p20_r0.txt	808	669	Greedy	FALSE	0.48	15.3
instancias\instancia_n60_p30_r0.txt	825	745	Greedy	FALSE	0.71	16.17
instancias\instancia_n75_p10_r0.txt	1430	1239	Greedy	FALSE	0.41	22.28
instancias\instancia_n75_p20_r0.txt	1192	863	Greedy	FALSE	1.3	18.19
instancias\instancia_n75_p30_r0.txt	754	616	Greedy	FALSE	1.08	14.69

Evaluación comparativa de Simulated Annealing y Greedy

Para analizar el rendimiento de los algoritmos desarrollados, se generaron 18 instancias con distintos tamaños ($n = 10$ a $n = 100$) y densidades ($p = 0.10, 0.20, 0.30$). En cada instancia se aplicaron dos algoritmos:

- **Greedy basado en densidad conflicto-aporte:** selecciona inversores priorizando el cociente entre su ganancia y el grado en el grafo de conflictos.
- **Simulated Annealing (SA)** sin búsqueda local: explora el espacio de soluciones aceptando vecinos peores con probabilidad decreciente según la temperatura.

Cambio de parámetros en Simulated Annealing

Para mejorar la capacidad exploratoria del algoritmo SA, se modificaron sus parámetros de la siguiente forma:

`python`

`T_0 = 5000` # Temperatura inicial

`M = 5000` # Número total de iteraciones

`C = 0.99` # Constante de enfriamiento

Esta configuración incrementa significativamente el número de pasos efectivos del algoritmo, permitiendo que explore con más libertad soluciones subóptimas en las primeras etapas, y se enfríe de forma más lenta y progresiva. Esto reduce el riesgo de quedar atrapado prematuramente en mínimos locales.

Resultados obtenidos

- **Greedy fue mejor en 13 de las 18 instancias.**
- **Simulated Annealing superó a Greedy en 3 instancias**, todas con $n \geq 50$ y $p \approx 0.20$, lo que sugiere que SA se adapta mejor a estructuras más densas o complejas.
- **En una instancia se observó empate exacto.**
- **Tiempos de ejecución:**
 - Greedy resolvió todas las instancias en menos de **1.5 ms**.
 - SA tardó entre **9 y 24 ms**, dependiendo del tamaño del grafo.

En instancias donde SA superó a Greedy, las diferencias en ganancia fueron del orden de **+25 a +130 unidades**, mientras que en los casos donde Greedy ganó, las diferencias también fueron significativas.

Conclusión final

El algoritmo Greedy demostró ser una solución altamente eficiente, rápida y confiable para este problema. Su comportamiento es consistente incluso en instancias grandes, y su velocidad lo hace ideal para entornos donde el tiempo de respuesta es crítico.

Por su parte, Simulated Annealing, si bien es más lento y no siempre supera a Greedy, logró mejores resultados en algunas instancias densas y complejas. Su flexibilidad y capacidad de escape de mínimos locales lo convierten en una buena alternativa cuando se dispone de tiempo adicional para optimizar la calidad de la solución.

En resumen, Greedy es la opción recomendada por defecto, especialmente para entornos en línea o en tiempo real. Simulated Annealing puede usarse como refinador en contextos offline o donde se prioriza la calidad de la solución por sobre la velocidad.

Parte 3: Autómatas Finitos

1. Construir un autómata finito determinista (AFD) de forma formal para cada uno de los lenguajes simples.

AFD de Lenguaje 1: $\{w \mid w \text{ tiene dentro suyo la subcadena "1011"}\}$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

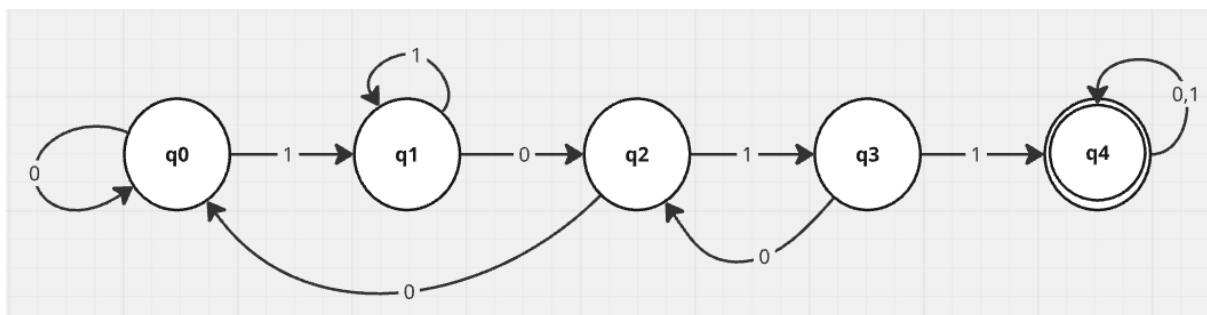
$$\Sigma = \{0, 1\}$$

$$q_0 = q_0$$

$$F = \{q_4\}$$

δ Corresponde a la siguiente tabla de transiciones

Estado	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_0	q_3
q_3	q_2	q_4
q_4	q_4	q_4



AFD de Lenguaje 2: $\{w \mid w \text{ tiene un número par de "0"}\}$

$$P = \{p_0, p_1\}$$

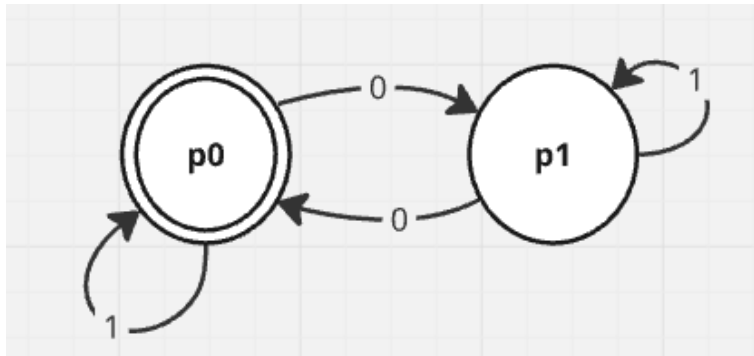
$$\Sigma = \{0, 1\}$$

$$p_0 = p_0$$

$$F = \{p_0\}$$

δ Corresponde a la siguiente tabla de transiciones

<i>Estado</i>	0	1
p_0	p_1	p_0
p_1	p_0	p_1



AFD de Lenguaje 3: $\{w \mid w \text{ toda posición impar es un "1"}\}$

$$Q = \{q_0, q_1, q_2\}$$

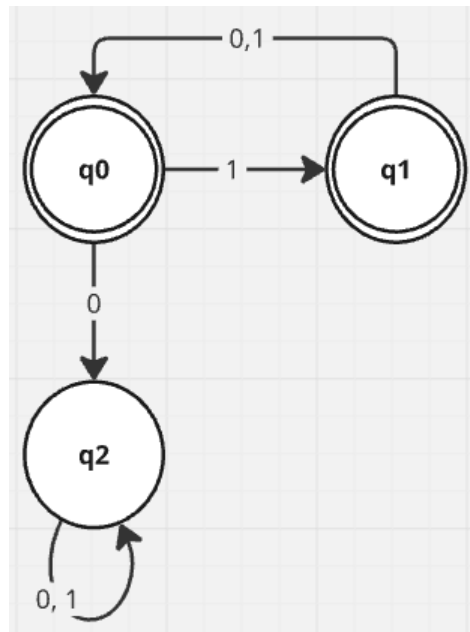
$$\Sigma = \{0, 1\}$$

$$q_0 = q_0$$

$$F = \{q_0, q_1\}$$

δ Corresponde a la siguiente tabla de transiciones

<i>Estado</i>	0	1
q_0	q_2	q_1
q_1	q_0	q_0
q_2	q_2	q_2



AFD de Lenguaje 4: $\{w \mid w \text{ termina con 3 "1"}\}$

$$P = \{p_0, p_1, p_2, p_3\}$$

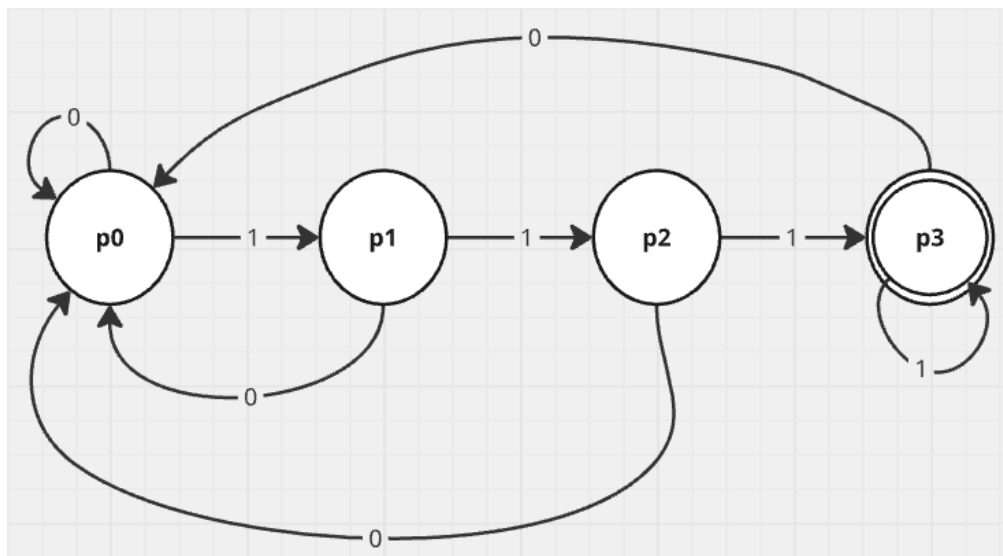
$$\Sigma = \{0, 1\}$$

$$p_0 = p_0$$

$$F = \{p_3\}$$

δ Corresponde a la siguiente tabla de transiciones

Estado	0	1
p_0	p_0	p_1
p_1	p_0	p_2
p_2	p_0	p_3
p_3	p_0	p_3



2. Basándose en los autómatas construidos generar de forma formal los dos autómatas finitos determinísticos de los lenguajes más complejos.

Siendo el lenguaje “a”: $\{w \mid w \text{ (tiene dentro suyo la subcadena “1011”) y (tiene un número par de “0”)}\}$, este lenguaje se define por la intersección de los siguientes dos lenguajes

- $a_1 = \{w \mid w \text{ (tiene dentro suyo la subcadena “1011”)}\}$
- $a_2 = \{w \mid w \text{ (tiene un número par de “0”)}\}$

Dado que ambos lenguajes son regulares, su intersección también lo es. El AFD resultante se obtiene como el producto cartesiano entre ambos a_1 y a_2 .

Si $a_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$, y $a_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ entonces el AFD del lenguaje **a** será tal que:

- $Q = Q_1 \times Q_2$

$$Q = \{(q_0, p_0), (q_0, p_1), (q_1, p_0), (q_1, p_1), (q_2, p_0), (q_2, p_1), (q_3, p_0), (q_3, p_1), (q_4, p_0), (q_4, p_1)\}$$

- $\Sigma = \{0, 1\}$
- $q_0 = (q_0, p_0)$
- $F = (q_4, p_0)$
- $\delta((q_i, p_j), \sigma) = (\delta_1(q_i, \sigma), \delta_2(p_j, \sigma))$ que corresponde a la siguiente tabla de transiciones:

Estado	0	1
(q_0, p_0)	(q_0, p_1)	(q_1, p_0)
(q_0, p_1)	(q_0, p_0)	(q_1, p_1)
(q_1, p_0)	(q_2, p_1)	(q_1, p_0)

(q1, p1)	(q2, p0)	(q1, p1)
(q2, p0)	(q0, p1)	(q3, p0)
(q2, p1)	(q0, p0)	(q3, p1)
(q3, p0)	(q2, p1)	(q4, p0)
(q3, p1)	(q2, p0)	(q4, p1)
(q4, p0)	(q4, p1)	(q4, p0)
(q4, p1)	(q4, p0)	(q4, p1)

Siendo el lenguaje “b”: $\{w \mid w \text{ (toda posición impar es un “1”) y (termina con 3 “1”)}\}$, este lenguaje se define por la intersección de los siguientes dos lenguajes

- $b_1 = \{w \mid \text{toda posición impar es un “1”}\}$
- $b_2 = \{w \mid w \text{ termina con 3 “1”}\}$

El AFD se puede obtener de la misma manera que el caso anterior, aplicando el producto cartesiano entre b_1 y b_2 .

Si $b_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$, y $b_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ entonces el AFD del lenguaje **a** será tal que:

- $Q = Q_1 \times Q_2$

$Q = \{(q_0, p_0), (q_0, p_1), (q_0, p_2), (q_0, p_3), (q_1, p_0), (q_1, p_1), (q_1, p_2), (q_1, p_3), (q_2, p_0), (q_2, p_1), (q_2, p_2), (q_2, p_3)\}$

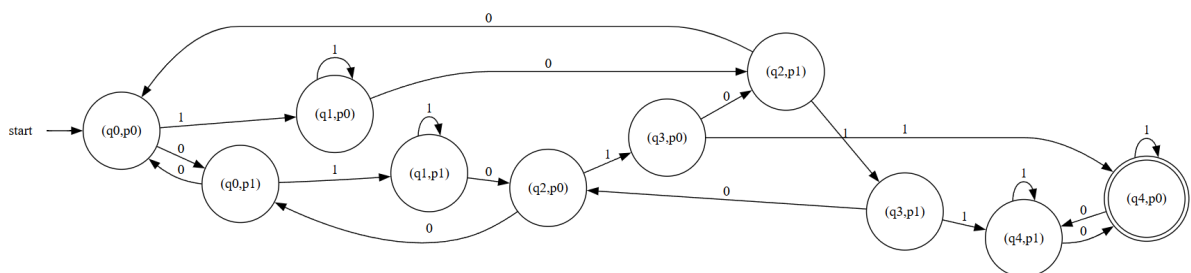
- $\Sigma = \{0, 1\}$
- $q_0 = (q_0, p_0)$
- $F = \{(q_0, p_3), (q_1, p_3)\}$
- $\delta((q_i, p_j), \sigma) = (\delta_1(q_i, \sigma), \delta_2(p_j, \sigma))$ que corresponde a la siguiente tabla de transiciones:

Estado	0	1
(q0, p0)	(q2, p0)	(q1, p1)
(q0, p1)	(q2, p0)	(q1, p2)
(q0, p2)	(q2, p0)	(q1, p3)
(q0, p3)	(q2, p0)	(q1, p3)
(q1, p0)	(q0, p0)	(q0, p1)
(q1, p1)	(q0, p0)	(q0, p2)

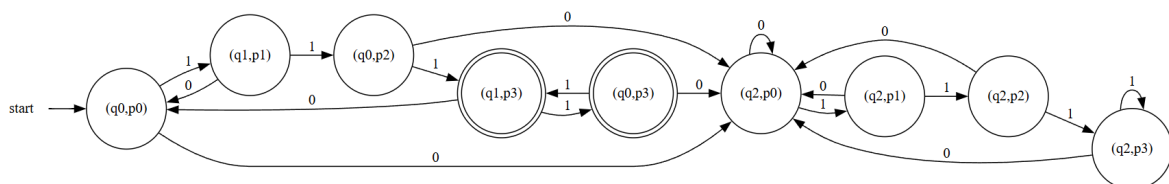
(q1, p2)	(q0, p0)	(q0, p3)
(q1, p3)	(q0, p0)	(q0, p3)
(q2, p0)	(q2, p0)	(q2, p1)
(q2, p1)	(q2, p0)	(q2, p2)
(q2, p2)	(q2, p0)	(q2, p3)
(q2, p3)	(q2, p0)	(q2, p3)

3. Representar cada uno de los AFD mediante su representación gráfica.

Representación del AFD que acepta el lenguaje “a”:



Representación del AFD que acepta el lenguaje “b”:



4. Para cada AFD Brindar un ejemplo de una cadena que acepta y otra que rechaza. Explicar cómo determinar su autómata si es aceptado o rechazado paso a paso.

Ejemplo aceptado para el lenguaje “a”: ‘10110100’

1. Contiene la cadena ‘1011’
2. la cantidad de ceros es par (4)

paso a paso

1. El estado inicial es (q_0, p_0)
2. Se lee cada símbolo y se sigue la transición de estados, como se detalla en los diagramas del punto 3, según el valor de cada símbolo

3. Una vez leída toda la cadena, si se llega al estado final (q_4, p_0) , la cadena es aceptada
transición de estados:

$$(q_0, p_0) \xrightarrow{1} \rightarrow (q_1, p_0) \xrightarrow{0} \rightarrow (q_2, p_1) \xrightarrow{1} \rightarrow (q_3, p_1) \xrightarrow{1} \rightarrow (q_4, p_1) \xrightarrow{0} \rightarrow (q_4, p_0) \xrightarrow{1} \rightarrow (q_4, p_0) \xrightarrow{0} \rightarrow (q_4, p_1) \xrightarrow{0} \rightarrow (q_4, p_0)$$

Una vez llegado el estado (q_4, p_1) ya reconocimos la cadena '1011' (y por ende la misma ya es una subcadena), y como se puede observar para el resto de la cadena: '0100':

- Al leer 0 ya alcanzamos la cantidad de ceros par y nos encontramos en el estado de aceptación.
- Al leer el 1 seguimos en el mismo estado con lo cual hasta aquí la cadena es aceptada.
- Finalmente leemos dos ceros más, preservando la cantidad par de ceros y quedando en el estado de aceptación.

Ejemplo rechazado para el lenguaje "a": '1010110'

1. contiene la cadena '1011'
2. la cantidad de ceros es impar (3)

Al llegar al final de la cadena, el estado correspondiente al autómata AFD no es el de aceptación. Por lo tanto, por más que se cumpla la primera restricción del lenguaje A, como la segunda no se cumple, el producto cartesiano entre a_1 y a_2 no llega a un estado final y la cadena es rechazada.

La transición de estados es la siguiente:

Para A1:

$$(q_0, p_0) \xrightarrow{1} \rightarrow (q_1, p_0) \xrightarrow{0} \rightarrow (q_2, p_1) \xrightarrow{1} \rightarrow (q_3, p_1) \xrightarrow{0} \rightarrow (q_2, p_0) \xrightarrow{1} \rightarrow (q_3, p_0) \xrightarrow{1} \rightarrow (q_4, p_0) \xrightarrow{0} \rightarrow (q_4, p_1)$$

En este caso no podemos observar que tras procesar "1010" cumplimos que la cantidad de ceros es par pero no nos encontramos en el estado de aceptación (nos falta contener la subcadena "1011"). Tras procesar los siguientes "1" consecutivos nos encontramos en el estado de aceptación pero aún nos falta procesar el "0" restante, esto nos deja en un estado que no es el de aceptación dado que finalizamos con la subcadena deseada pero con una cantidad impar de ceros.

Ejemplo aceptado para el lenguaje "b": '1011111'

1. todas las posiciones impares valen 1
2. la cadena finaliza en '111'

paso a paso

Si bien tras leer el primer "1" se cumple que todas las posiciones impares valen "1", aún no se cumple que la cadena finaliza en "111" por lo tanto es correcto no encontrarnos en un estado de aceptación. Posteriormente leemos "01" preservando que todas las posiciones impares valen "1" pero al igual que antes no nos encontramos en un estado de aceptación. Luego leemos otro "1" y se repite la situación pero al leer el próximo "1" si nos encontramos en uno de los estados de

aceptación (hasta ahora procesamos “10111” y falta procesar “11”). De aquí en más restan leer “11” y se observa fácilmente que quedaremos en uno de los estados de aceptación, cumpliendo ambas condiciones.

$$(q_0, p_0) \xrightarrow{1} (q_1, p_1) \xrightarrow{0} (q_0, p_0) \xrightarrow{1} (q_1, p_1) \xrightarrow{1} (q_0, p_2) \xrightarrow{1} (q_1, p_3) \xrightarrow{1} (q_0, p_3) \xrightarrow{1} (q_1, p_3)$$

Finalmente llegamos a uno de los estados de aceptación.

Ejemplo rechazado para el lenguaje “b”: ‘1110101’

1. La cadena termina en ‘101’.

paso a paso

Podemos observar que al leer los primeros tres símbolos (“111”) se cumplen ambas condiciones y nos encontramos en un estado final, aún así restan símbolos por procesar. Tras leer el “0” ya no nos encontramos en un estado de aceptación lo cual es correcto porque la cadena no estaría terminando en “111” sino que en “110”. Continuamos procesando un “1” y si bien cumplimos con “1” en las posiciones impares nos encontramos en un estado que no es de aceptación. Posteriormente procesamos “0”, seguimos cumpliendo la condición mencionada anteriormente pero no la restante y tras leer el último símbolo “1” tampoco nos encontramos en un estado de aceptación por la misma razón.

$$(q_0, p_0) \xrightarrow{1} (q_1, p_1) \xrightarrow{1} (q_0, p_2) \xrightarrow{1} (q_1, p_3) \xrightarrow{0} (q_0, p_0) \xrightarrow{1} (q_1, p_1) \xrightarrow{0} (q_0, p_0) \xrightarrow{1} (q_1, p_1)$$

Bibliografía

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introduction to automata theory, languages, and computation (3rd ed.). Pearson Education.

Mishra, A. (9 jul 2021). *Understanding Branch and Bound in Optimization Problems*. Walmart Global Tech en Medium. Recuperado de <https://medium.com/walmartglobaltech/understanding-branch-and-bound-in-optimization-problems-d8117da0e2c5>

Taylor, B. (2007). *Module C - Integer Linear Programming and Branch & Bound*. Instituto Superior Técnico. Recuperado de https://web.tecnico.ulisboa.pt/mcasquilho/compute/_linpro/TaylorB_module_c.pdf